

SRE-GitOps设计

通过GitOps的设计与落地，来满足云原生环境下业务的持续交付，建立标准的开发、测试、生产环境，形成统一的规范，完善自动化，为业务快速交付赋能。



整体架构

（蓝色代表后续做ARM兼容适配的地方）

[请至钉钉文档查看「流程图」](#)

基础环境

- IDC机房
 - AMD服务器
 - ARM服务器（资源采购&预算）
 - 存储服务
- 域名
- 证书

Kubernetes

运行环境支持AMD和ARMv8

部署架构图

原则：**集群隔离**。一个环境一个集群，互不干扰。

目前已有四套集群，提供给研发、测试、生产以及Hotfix、poc使用。

预估信创适配需3-4套ARM集群环境。

后续共需管理7-8个集群环境

[请至钉钉文档查看「流程图」](#)

运维管理

代码仓库

通过流程管理，进行仓库的开通或变更，以产品组进行划分和管理，建立起唯一授信源，为后续Pipeline设计做好基础建设

[JKStack代码仓库](#)



精鲲科技-JKStack 部门标准化仓库 Root 组

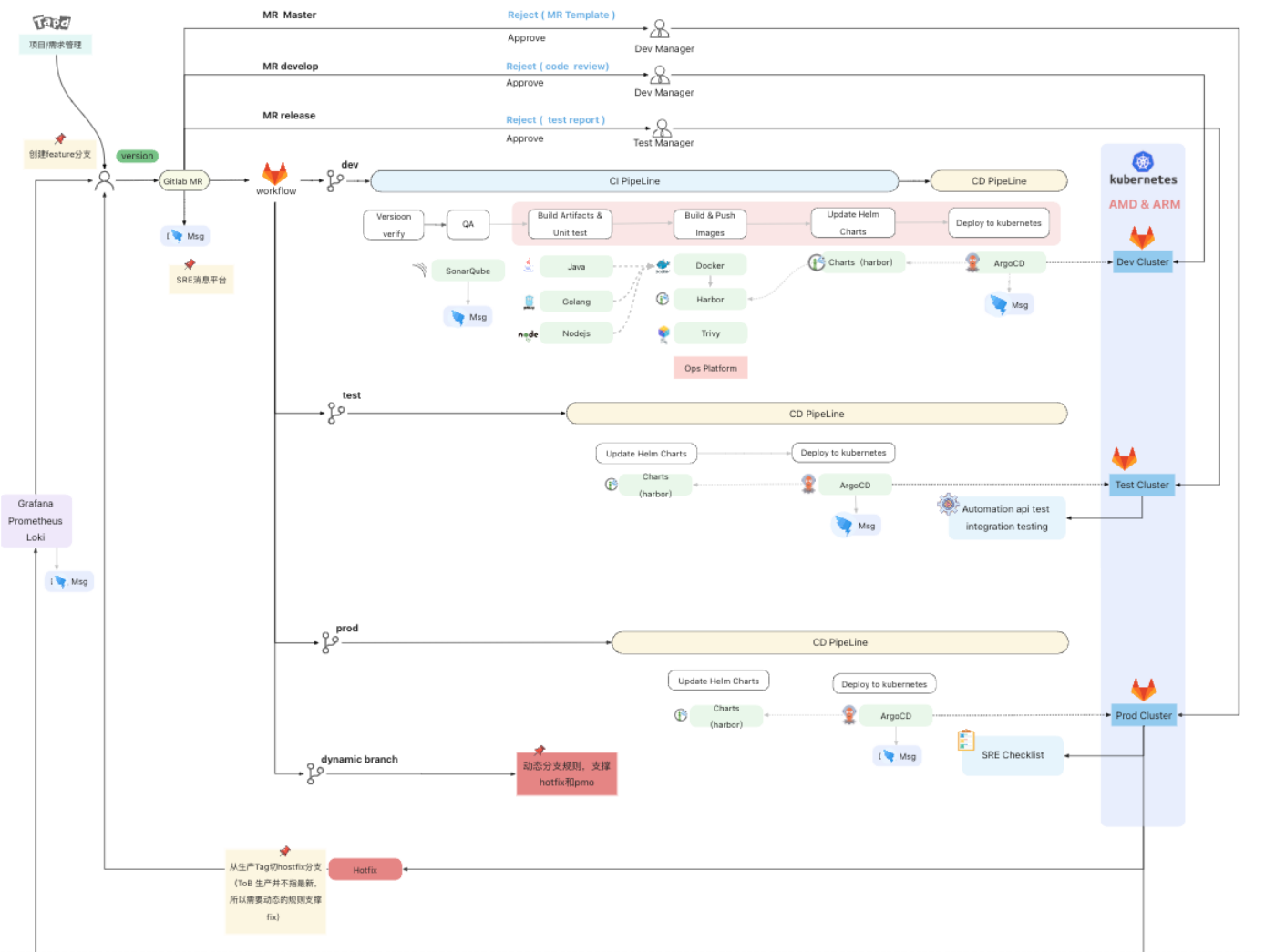
Subgroups and projects		Shared projects	Archived projects	Search by name		Last created	
>	 B bd 				 0	 1	 2
>	 D df 				 1	 4	 7
>	 N ndi 				 1	 4	 4
>	 C common  公共服务				 1	 3	 5
>	 D dsp 				 0	 4	 10
>	 S samp 				 0	 3	 3
>	 A aiops 				 0	 2	 3
>	 A architecture  Owner 架构组-pubulic库			 	 0	 2	 3
>	 P PMO 				 3	 7	 3
>	 D dp  统一架构&系统服务 源码仓库				 3	 0	 10
>	 D delivery 				 1	 1	 4
>	 Q qa  测试自动化代码仓库				 2	 15	 18
>	 S sre 				 2	 5	 6
>	 I infra  基础设施组标准化源码仓库				 0	 4	 1
>	 D dv  dv 产品线源码仓库				 0	 3	 1
>	 C cmdp  Owner cmdp 产品线仓库组			 	 0	 4	 5
>	 D dsm  Owner dsm 产品线源码仓库组			 	 0	 6	 8
>	 di  Owner			 			

GitOps

原则： 采用**一切皆代码的原则，进行基础服务和业务项目的维护管理。**

流程设计

从代码提交到项目上线全流程



仓库规划

运维工程项目**规划**

D

devOps

Project ID: 566

416 Commits

10 Branches

0 Tags

2.8 MB Files

3.9 MB Storage

master

devops /

+

History

Find file

Web IDE

Clone

enablej df ingress

lizili authored 1 week ago

1b60ff88

README

CI/CD configuration

Add LICENSE

Add CHANGELOG

Add CONTRIBUTING

Add Kubernetes cluster

Name	Last commit	Last update
cicd	refactor gitlab ci templates	2 weeks ago
gitops	enablej df ingress	1 week ago
misc	add test-mysql svc config	1 month ago
playbook/rke	add uat k8s cluster	1 month ago
templates	update template .vue.yaml	5 months ago
utils	fix longhorn util config	1 month ago
.gitattributes	add .gitattributes	1 year ago
.gitignore	Feature/version	1 year ago

搜索文件

cicd

gitops

misc

playbook

templates

utils

.gitattributes

.gitignore

.gitlab-ci.yml

README.md

VERSION

目录说明：

- cicd**：用来做产品的打包和部署管理，产线内gitlab-ci直接引用即可，做到研发和运维代码管理上的解耦

.gitlab-ci.yml

133 Bytes

Edit

Web IDE

Pipeline Editor

Replace

Delete

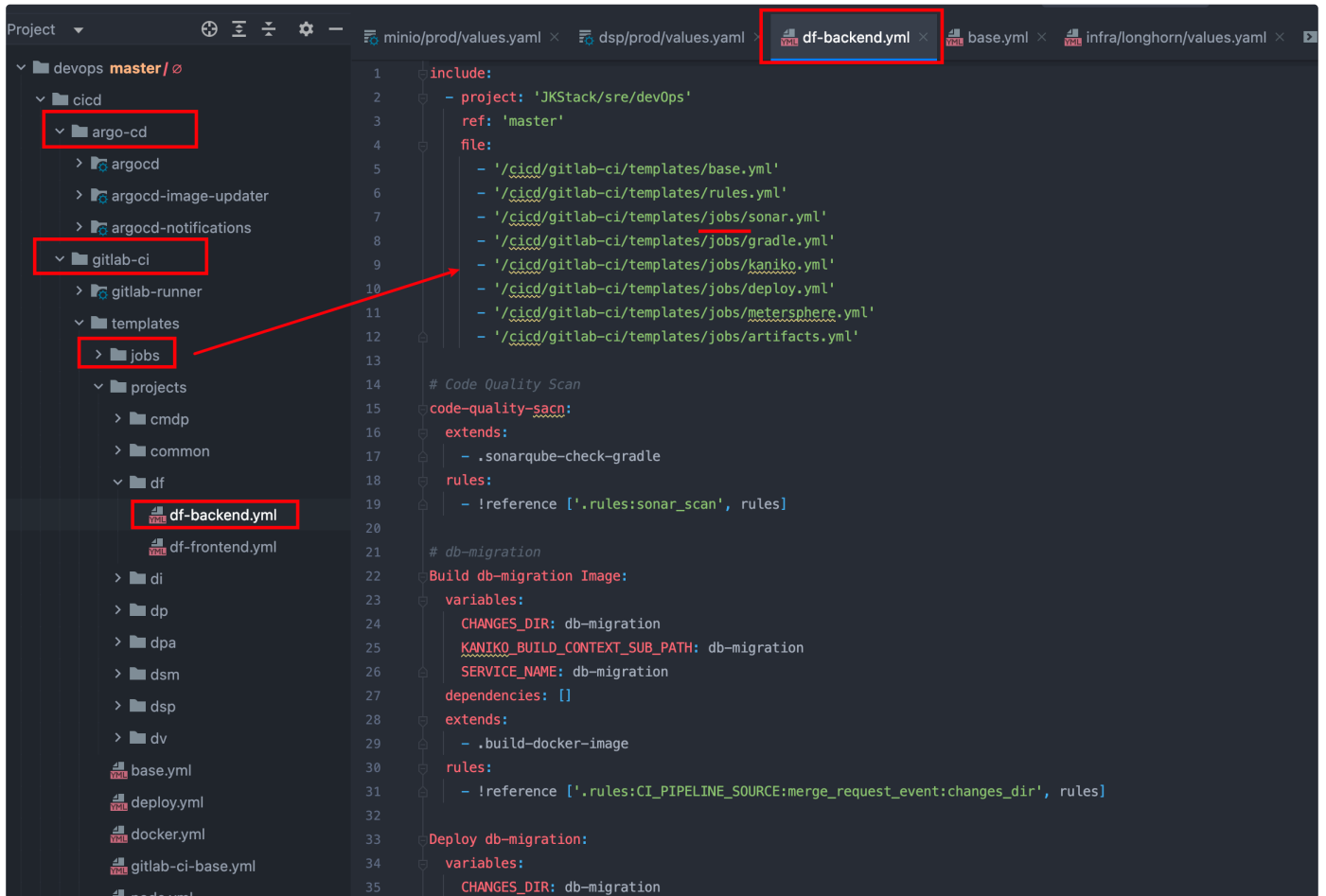
```
1 include:
2   - project: "JKStack/sre/devOps"
3     ref: master
4     file:
5       - '/cicd/gitlab-ci/templates/projects/df/df-backend.yml'
6
```

1. gitlab-ci.yaml文件

- gitlab规定的固定命名文件，用来触发git pipeline，其内容编写遵循gitlab的编排语法，作用类似jenkinsfile
- gitops**：用来做运维管理，如基础设施、中间件等
- misc**：试验性的临时功能。未集成到标准编排中的尝试性功能，均在此处编排和管理
- playbook**：Ansible剧本编排服务，目前提供基于rke工具拉起Kubernetes集群功能（不支持arm）
- template**：git pipeline 通用模板文件，现已弃用，迁移至cicd中
- utils**：运维工具、程序管理，如mysql备份任务，kubernetes调试工具等

目录详解

cicd



1. gitlab-ci: 项目的持续集成管理（ARM环境的制品产出，主要是在这里作适配）

1. gitlab-runner: ci任务的执行者，负责 ci 编排文件的处理和脚本的执行
2. template: CI编排文件
 1. job: 提供通用基础服务，如gradle、maven、golang、artifacts等pipeline节点服务
 2. projects: 按产线进行归类，按服务进行单独的编排管理，针对不同的服务，可在此进行模板的引用和特性的扩展

ci

制品构建的所有操作均在此处调整，如和产线协商的变量新增(XC_TONG_WEB_ENABLED)以区分东方通 jar依赖等，通过逻辑判断和处理，产出不同的制品。

gradle.yml 1.94 KB

Edit Web IDE Replace Delete

```

1 .gradle-assemble:
2 stage: package
3 variables:
4   GRADLE_OPTS: "-Dorg.gradle.daemon=false"
5   image: ${REGISTRY_HOST}/cicd/gradle:${PACKAGE_GRADLE_DOCKER_VERSION} # 6.5.1-jdk8
6   before_script:
7     - |
8       if [[ -d "${PROJECT_SUB_PATH:-}" ]]; then
9         cd "${PROJECT_SUB_PATH}"
10      fi
11     - |
12       export GRADLE_USER_HOME=`pwd`/.gradle
13       if [[ -z "${MODULE:-}" ]]; then
14         echo "MODULE variable is not specific, will assemble entire project."
15       else
16         export MODULE=$(echo "$MODULE" | tr / : | sed 's/^:/' | sed 's$/:/')
17         echo "$MODULE will be assemble using gradle."
18      fi
19 script:
20   - "gradle --build-cache ${MODULE}build -x test"
21   - |
22     export ZIPPED_ARCHIVE_API="${CI_API_V4_URL/projects/${CI_PROJECT_ID}/jobs/${CI_JOB_ID}/artifacts"
23     echo "ZIPPED_ARCHIVE_API: ${ZIPPED_ARCHIVE_API}"
24     echo "ZIPPED_ARCHIVE_API=${ZIPPED_ARCHIVE_API}" > artifacts-${CI_JOB_ID}.env
25 cache:
26   key: "${CI_JOB_NAME}"
27   paths:
28     - .gradle
29 artifacts:
30   when: on_success
31   expire_in: 2 days
32   paths:
33     - ${CI_PROJECT_DIR}/**/build/libs/*.jar
34   reports:
35     dotenv: ${CI_PROJECT_DIR}/**/artifacts-${CI_JOB_ID}.env
36

```

新增 XC_TONG_WEB_ENABLED

cicd

argo-cd

gitlab-ci

gitlab-runner

templates

jobs

argocd-deploy.yml

artifacts.yml

deploy.yml

go.yml

gradle.yml

kaniko.yml

maven.yml

metersphere.yml

node.yml

sonar.yml

projects

cmdp

cmdb-front.yml

cmdb.yml

cmdp-front-h5.yml

jkstack-cmdp.yml

common

df

di

dp

dpa

dsm

dsp

dv

base.yml

如何产出多环境下的制品？

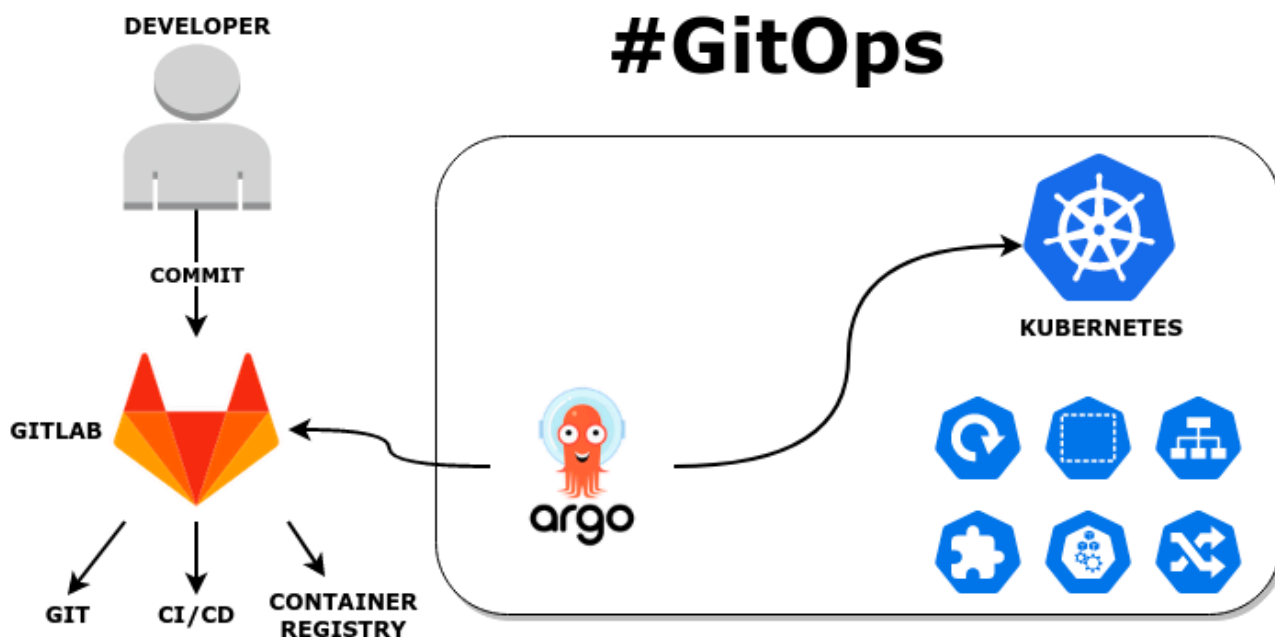
需调度runner到不同架构下的集群节点，进行编排文件的执行，这块逻辑会在gitlab-runner中进行兼容处理，并在 jobs 中做逻辑处理

cd

如何发布到不同的环境实行多环境的？

目前的发布是通过ArgoCD进行操作

- argocd是基于kubernetes的声明式的CD工具



通过argo应用的创建和声明，来监控编排文件的变化，从而实现产品的CD操作

JKStack > sre > devOps > Repository

master

devops / .. / products / dev-df.yaml

Find file

Blame

History

Permalink

 add new argocd managed product df

Johnny Zhang authored 2 months ago

3148fbe6

dev-df.yaml 461 Bytes

Edit

Web IDE

Replace

Delete







1

apiVersion: argoproj.io/v1alpha1

2

kind: Application

3

metadata:

4

name: dev-df

5

namespace: argocd

6

labels:

7

app: dev-df

8

spec:

9

destination:

10

namespace: df

11

server: https://k8s-dev.jkstack.org:6443

12

project: df

13

source:

14

repoURL: http://gitlab.jkstack.org/jkstack/sre/jkstack-apps.git

15

targetRevision: develop

16

path: df/dev

17

syncPolicy:

18

automated:

19

prune: true

20

selfHeal: true

21

syncOptions:

22

- CreateNamespace=true

cicd

argo-cd

templates

applications

dev

infra

middleware

products

dev-df.yaml

dev-jkexec.yaml

dev-jkflow.yaml

dev-jkstack-cmdp.yaml

dev-jkstack-di.yaml

dev-jkstack-dp.yaml

dev-jkstack-dpa.yaml

dev-jkstack-dsm.yaml

dev-jkstack-dsp.yaml

dev-jkstack-dv.yaml

dev-jkstack-license.yaml

dev-jkstack-samp.yaml

prod

test

uat

clusters

projects

repositories

Chart.yaml

Argo 下产品应用服务编排 （ARM的部署，在这里作适配）

- 命名的区分
- 运行环境指定
 - 可区分不同的环境发布到不同的集群内，达到多环境的隔离
- 编排文件指定
 - 可以指定应用的编排文件，后续在编排中通过申请的通配域名，即可实现多环境的访问切换，如
 - https://jkstack.prod.jingkunsystem.com:29039/
 - https://jkstack.test.jingkunsystem.com:29040/

Applications

2.4.8+e

+ NEW APP

SYNC APPS

REFRESH APPS

df

Log out

FILTERS

★ Favorites Only

SYNC STATUS

Unknown 0

Synced 36

OutOfSync 7

HEALTH STATUS

Unknown 0

Progressing 1

Suspended 0

Healthy 34

dev-df

Project: df

Labels: app=dev-df, argocd.argoproj.io/instance...

Status: Healthy Synced

Repository: http://gitlab.jkstack.org/jkstack/sre/jk...

Target R...: develop

Path: df/dev

Destinati...: https://k8s-dev.jkstack.org:6443

Namesp...: df

SYNC

REFRESH

DELETE

prod-df

Project: df

Labels: app=prod-df, argocd.argoproj.io/instanc...

Status: Healthy Synced

Repository: http://gitlab.jkstack.org/jkstack/sre/jk...

Target R...: master

Path: df/prod

Destinati...: https://kubernetes.default.svc

Namesp...: df

SYNC

REFRESH

DELETE

test-df

Project: df

Labels: app=test-df, argocd.argoproj.io/instance...

Status: Healthy Synced

Repository: http://gitlab.jkstack.org/jkstack/sre/jk...

Target R...: release

Path: df/test

Destinati...: https://k8s-test.jkstack.org:6443

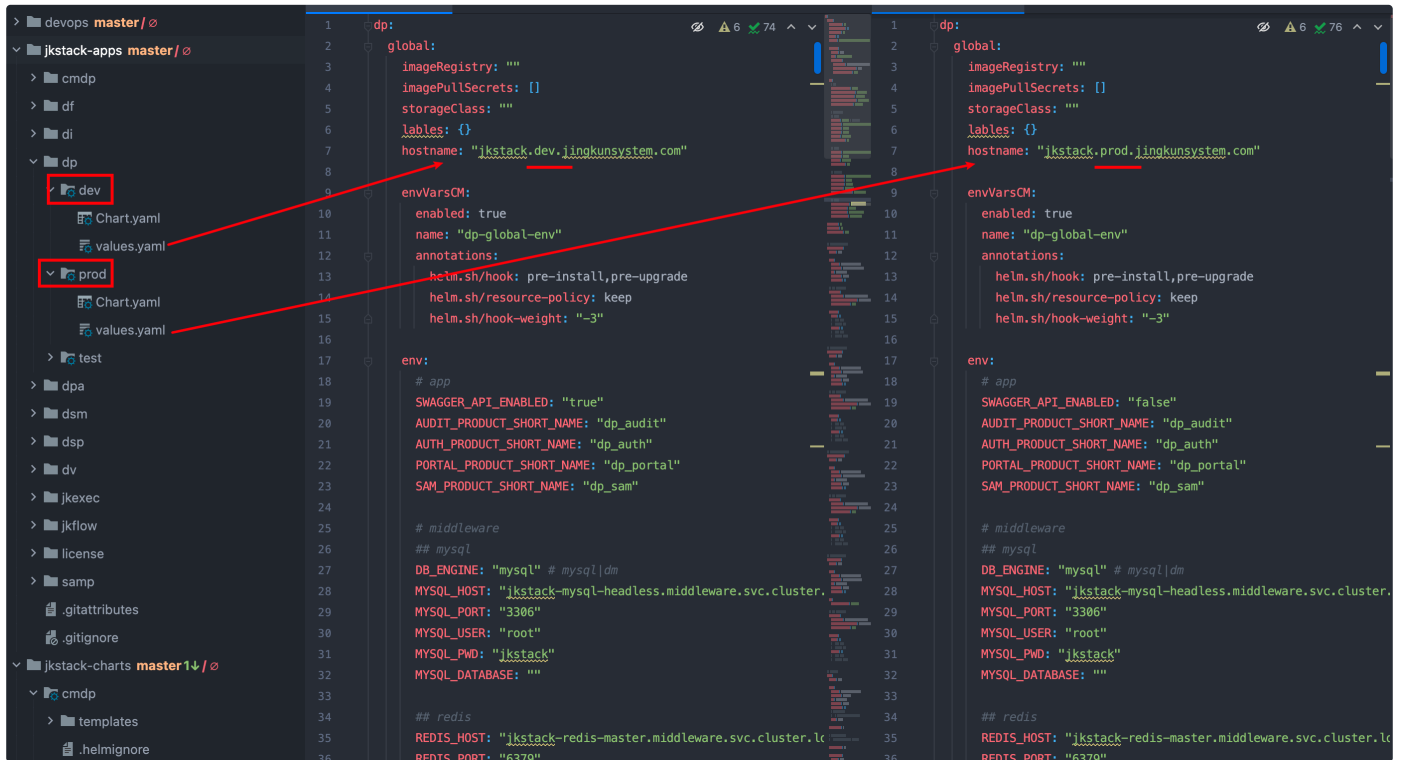
Namesp...: df

SYNC

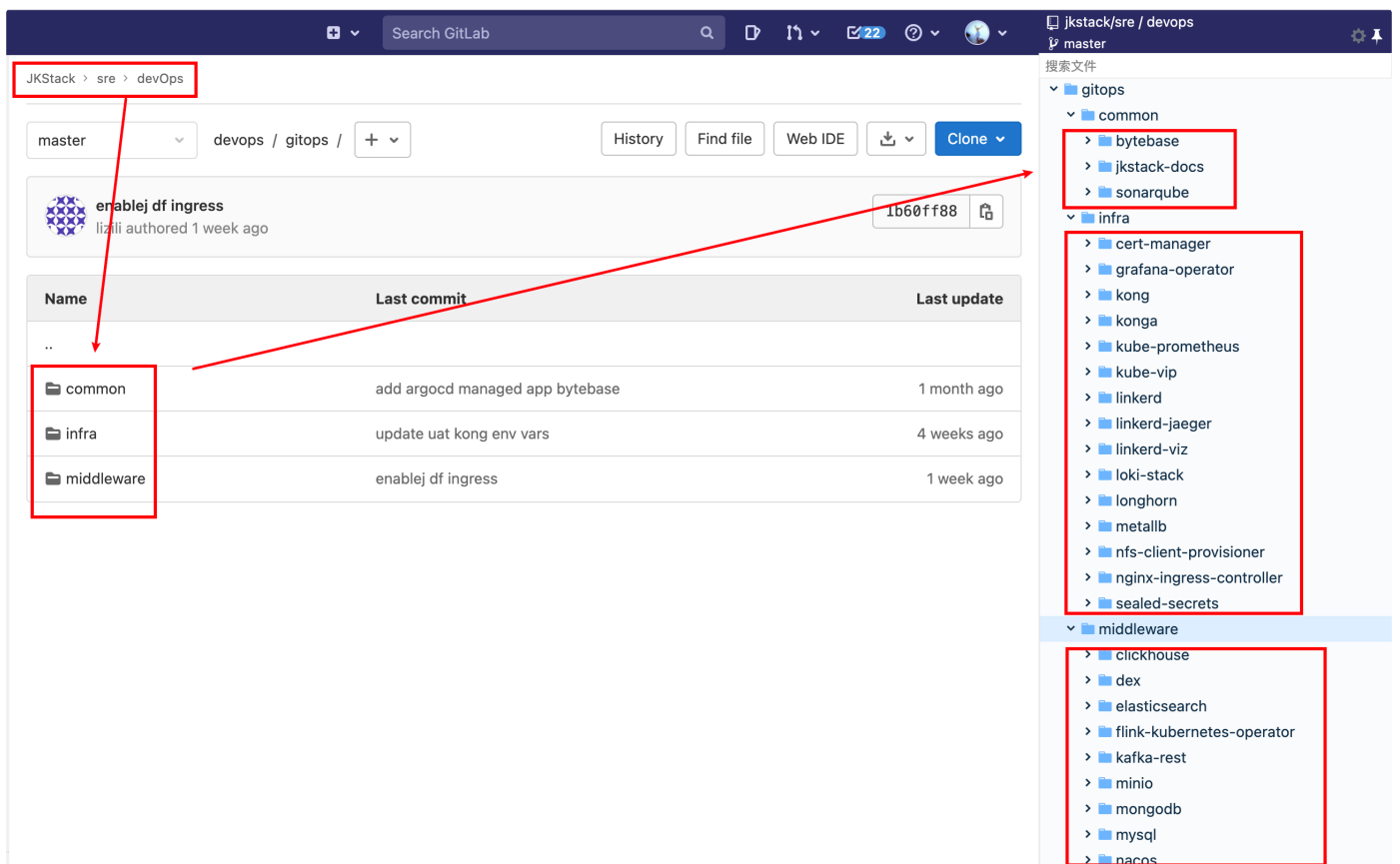
REFRESH

DELETE

编排文件对应的就是ArgoCD声明的仓库路径，从而实现了域名和环境的区分，产品的发布只需要修改这里即可，这样后续在pipeline中就可以实现自动化



gitops



ARM运行环境，主要是在这里作适配

1. common: 公共服务管理，比如SQL审计平台 bytebase，研发文档服务docs 等
2. infra: 基础设施服务，和kubernetes集群相关，如SLB (metallb)，存储服务
3. middleware: 中间件服务，产品依赖中间件的编排均在此维护

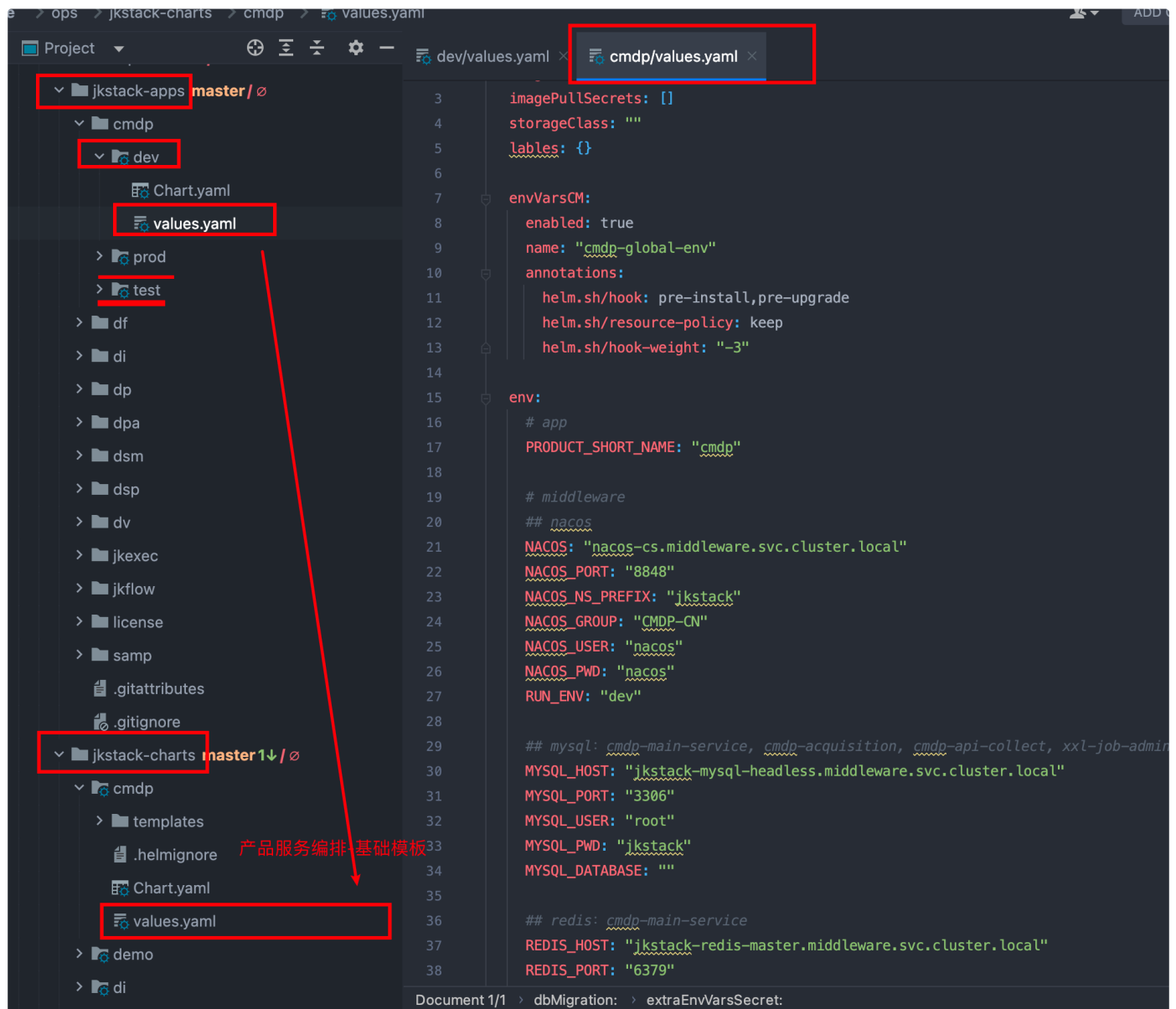
4. jkstack-charts （项目外单独维护）

1. 按产线归类，所有的产品运行编排服务均在此仓库进行维护和管理

5. jkstack-apps （项目外单独维护）

1. 产品运行环境归类，基于业务编排模板，进行差异化修改和管理，之后可由Argo-cd发布到对应的集群环境中

charts和app介绍



如图

apps中的服务按环境进行区分管理，通过差异化配置供ArgoCD使用，从而发布到不同环境。其核心编排均来自服务编排模板，优势在于

- 避免多环境操作同一个文件，管理混乱
- 避免多环境同时发布，同文件的冲突
- 工程文件和基础环境保持了一致
- 文件改动少，便于自动化

其他目录

和cicd关系不大，暂不详细介绍

镜像管理

通过harbor管理所有的编排文件和制品镜像 （Harbor需升级用来支持ARM制品的保存）

版本上报

```
    echo "IMAGE_TAG: ${IMAGE_TAG}"
    echo "DEPLOY_ENV: ${DEPLOY_ENV}"
    echo "ZIPPED_ARCHIVE_API: ${ZIPPED_ARCHIVE_API}"
    echo "AUTOOPS_URL: ${AUTOOPS_URL}"
else
    echo 'Fatal: "artifacts:reports:dotenv" variables PRODUCT_VERSION or IMAGE_TAG is null.'
    exit 1
fi
if [[ "${DEPLOY_ENV}" = "prod" ]] || [[ "${DEPLOY_ENV}" = "uat" ]];then
    echo "Updating ${DEPLOY_ENV} artifacts info to platform."
else
    echo "Would not update ${DEPLOY_ENV} artifacts info to platform."
    exit 0
fi
script:
- |
    # set -x
    # apk add curl
    AUTOOPS_SCHEME=$(echo "${AUTOOPS_URL}" | awk -F: '{print $1}')
    AUTOOPS_HOST=$(echo "${AUTOOPS_URL}" | sed -n -r 's#(http[s]?://)?(.*?\.?+\.{2,3})(:.)?(/.)?#\2#p')
    AUTOOPS_PORT=$(echo "${AUTOOPS_URL}" | sed -n -r 's#.+:[0-9]+(/.)?#\1#p')
    if [[ -n "${AUTOOPS_PORT}" ]]; then
        AUTOOPS_PORT=${AUTOOPS_PORT}
    elif [[ "${AUTOOPS_SCHEME}" == "https" ]]; then
        AUTOOPS_PORT=443
    else
        AUTOOPS_PORT=80
    fi
    if nc -z -w 5 ${AUTOOPS_HOST} ${AUTOOPS_PORT}; then
        if [[ -n "${ZIPPED_ARCHIVE_API}" ]]; then
            count=0
            until [[ $count -ge 5 ]]; do
                # ((count++))
                count=$((count + 1))
                echo "Updating ${PRODUCT_NAME} ${SERVICE_NAME} zipped archive package info to autoops platform the $count time"
                if curl -X POST \
                    -H "accept: application/json" \
                    -H "Content-Type: application/json" \
                    -d "{ \"product_name\": \"${PRODUCT_NAME}\", \"product_version\": \"${PRODUCT_VERSION}\", \"product_image_tag\": \"${IMAGE_TAG}\", \"product_deploy_env\": \"${DEPLOY_ENV}\" }" \
                    "${AUTOOPS_URL}"
```

搜索文件

- cicd
 - argo-cd
 - gitlab-ci
 - gitlab-runner
 - templates
 - jobs
 - argocd-deploy.yml
 - artifacts.yml
 - deploy.yml
 - go.yml
 - gradle.yml
 - kaniko.yml
 - maven.yml
 - metersphere.yml
 - node.yml
 - sonar.yml
 - projects
 - base.yml
 - deploy.yml
 - docker.yml
 - gitlab-ci-base.yml
 - node.yml
 - rules.yml
 - sonar.yml
 - gitops
 - misc
 - playbook
 - templates
 - utils
 - .gitattributes
 - .gitignore

制品上传

```

cat ${SERVICE_NAME}-deploy.env
- echo "Building and shipping image from ${KANIKO_BUILD_CONTEXT_PATH} to ${REGISTRY_HOST}/${REGISTRY_REPO}"
- mkdir -p /kaniko/.docker
- echo "${auth}":{"${REGISTRY_HOST}":{"auth":"${(echo -n ${REGISTRY_USER}:${REGISTRY_PWD} | base64 |
- DOCKERFILE_PATH=${DOCKERFILE_PATH:-"${KANIKO_BUILD_CONTEXT_PATH}/Dockerfile"}
- |
# KANIKO_BUILD_ARGS="--build-arg QA_PLATFORM_ADDRESS=${QA_PLATFORM_ADDRESS}
#
# --build-arg SERVICE_NAME=${SERVICE_NAME}
#
# --build-arg INCLUDES_PACKAGE=${INCLUDES_PACKAGE}
#
# --build-arg PROJECT_URL=${CI_PROJECT_URL}
#
# --build-arg PROJECT_COMMIT_REF_NAME=${CI_COMMIT_REF_NAME}
#
# --build-arg PROJECT_COMMIT_SHORT_SHA=${CI_COMMIT_SHORT_SHA}"
if [[ "${CI_MERGE_REQUEST_TARGET_BRANCH_NAME}" == "${JKSTACK_PROD_BRANCH_NAME}" ]]; then
/kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
--skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
--destination ${REGISTRY_HOST}/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG_LATEST} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/$SERVICE_NAME:${TAG_BRANCH} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/$SERVICE_NAME:${TAG} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/${SERVICE_NAME}:current-${DEPLOY_ENV:-latest} \
--destination $REGISTRY_HOST/$REGISTRY_REPO_DEFAULT/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG_LATEST} \
--destination $REGISTRY_HOST/$REGISTRY_REPO_DEFAULT/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG_BRANCH} \
--destination $REGISTRY_HOST/$REGISTRY_REPO_DEFAULT/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG}
elif [[ "${CI_MERGE_REQUEST_TARGET_BRANCH_NAME}" == "${JKSTACK_RELEASE_BRANCH_NAME}" ]]; then
/kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
${KANIKO_BUILD_ARGS} \
--skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
--destination ${REGISTRY_HOST}/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG_BRANCH} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/$SERVICE_NAME:${TAG} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/$SERVICE_NAME:current-${DEPLOY_ENV:-latest}
elif [[ "${DEPLOY_ENV}" == "uat" ]]; then
/kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
--skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \
--destination ${REGISTRY_HOST}/${REGISTRY_REPO}/${SERVICE_NAME}:${TAG_BRANCH} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/$SERVICE_NAME:${TAG} \
--destination $REGISTRY_HOST/$REGISTRY_REPO/$SERVICE_NAME:current-${DEPLOY_ENV:-latest} \
--destination $REGISTRY_HOST/$REGISTRY_REPO_DEFAULT/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG_BRANCH} \
--destination $REGISTRY_HOST/$REGISTRY_REPO_DEFAULT/${PRODUCT_NAME}/${SERVICE_NAME}:${TAG}
else
/kaniko/executor --context ${KANIKO_BUILD_CONTEXT_PATH} --dockerfile ${DOCKERFILE_PATH} \
--skip-tls-verify --skip-tls-verify-pull --insecure --insecure-pull \

```

搜索文件

- ▼ cicd
 - > argo-cd
 - ▼ gitlab-ci
 - > gitlab-runner
 - ▼ templates
 - ▼ jobs
 - Y argocd-deploy.yml
 - Y artifacts.yml
 - Y deploy.yml
 - Y go.yml
 - Y gradle.yml
 - Y kaniko.yml**
 - Y maven.yml
 - Y metersphere.yml
 - Y node.yml
 - Y sonar.yml
 - > projects
 - Y base.yml
 - Y deploy.yml
 - Y docker.yml
 - Y gitlab-ci-base.yml
 - Y node.yml
 - Y rules.yml
 - Y sonar.yml
- > gitops
- > misc
- > playbook
- > templates
- > utils
 - ◆ .gitattributes
 - ◆ .gitignore

规则:

1. jkstack/<产品>/服务名:版本 (jkstack-charts也在此项目中)
2. middleware/[registry]/服务名:版本
3. infra/[分类]/服务名:版本

☐
jkstack/dp/db-migration

☐
jkstack/dp/authz

☐
infra/longhornio/longhorn-manager

☐
infra/longhornio/longhorn-engine

☐
infra/bitnami/nginx

☐
infra/bitnami/nginx-ingress-controller

☐
middleware/bitnami/mysql

☐
middleware/bitnami/minio

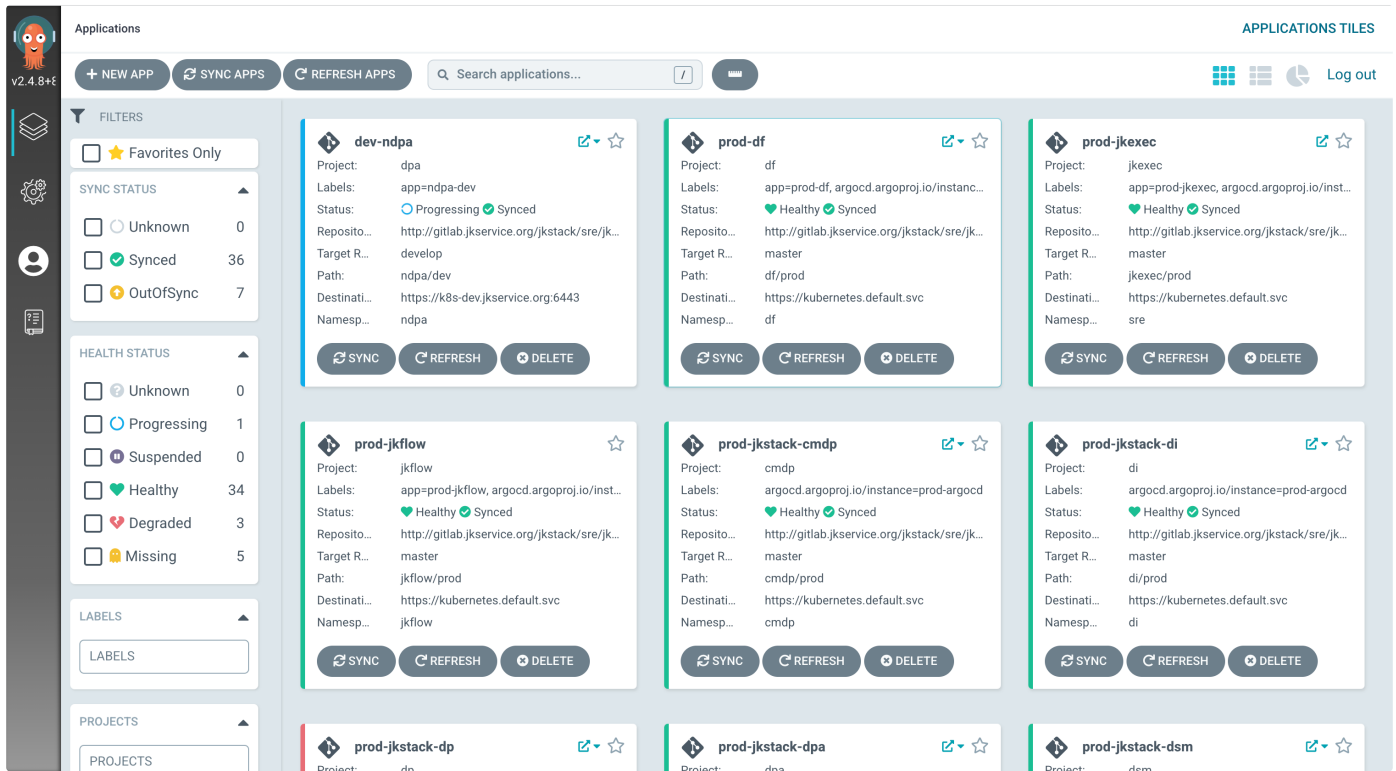
流水线

具备以上条件后后，通过git pipeline 即可将产品的CICD串联起来，实现了多环境的发版控制

Status	Pipeline ID	Triggerer	Commit	Stages	Duration	
passed	#45077 latest		feat/102897... 1f04080f Merge branch 'develop' i...	✓	00:01:56 1 week ago	⋮
passed	#45075 latest detached		1028 56295a42 Feat/1028978 xc 租房信...	✓✓✓✓»	00:02:35 1 week ago	⋮
passed	#45057 latest detached		1036 19cd62df update: 租房信息上报使...	»✓✓✓✓	00:01:52 1 week ago	▶▼⋮
passed	#45055 detached		1036 17c0d3f9 Merge branch 'develop' i...	»✓✓✓✓	00:01:59 1 week ago	▶▼⋮

Scan	Test	Package	Build	Deploy	Artifacts
code-qualit... scan	Unit Test alarm... test	Package alarm... package	Build alarm-ma... build	Deploy alarm-... deploy	Uploa
	Unit Test dataf... test	Package dataf... package	Build config-se... build	Deploy config-... deploy	Uploa
	Unit Test dataf... test	Package dataf... package	Build dataflow-... build	Deploy dataflo... deploy	Uploa
	Unit Test dataf... test	Package dataf... package	Build dataflow-... build	Deploy dataflo... deploy	Uploa
	Unit Test dataf... test	Package dataf... package	Build dataflow-... build	Deploy dataflo... deploy	Uploa
	Unit Test debe... test	Package debe... package	Build dataflow-... build	Deploy dataf... deploy	Uploa
	Unit Test di-co... test	Package di-co... package	Build dataflow-... build	Deploy db-... deploy	Uploa
			Build db-migra... build		





自动化工具

略（需做ARM适配）

1. 版本管理平台
2. 部署客户端（可将产品版本依赖及镜像一键离线、导入导出，并拉起集群环境）


```
cannot connect to the docker daemon at unix:///var/run/docker.sock, is the docker daemon running?  
x ⚡ root@zilipydev ~ devops-manager-client init -c /etc/devops-manager-client/config.yaml  
2023-01-20T14:09:00.648+0800 INFO cmd/init.go:35 Init command  
2023-01-20T14:09:00.650+0800 INFO workflow/system.go:11 操作系统初始化开始  
SELINUX=Disabled and will next reboot to disabled  
firewalld is stopping  
firewalld was disabled  
fs.inotify.max_user_watches = 524288  
net.ipv4.ip_forward = 1  
net.bridge.bridge-nf-call-arptables = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
net.bridge.bridge-nf-call-iptables = 1  
net.ipv4.ip_local_reserved_ports = 30000-32767  
vm.max_map_count = 262144  
vm.swappiness = 1  
fs.inotify.max_user_instances = 524288  
kernel.pid_max = 65535  
2023-01-20T14:09:15.793+0800 INFO workflow/system.go:18 操作系统初始化结束  
  
[Step 0]: checking if docker is installed ...  
  
Note: docker version: 20.10.17  
  
[Step 1]: checking docker-compose is installed ...  
  
Note: docker-compose version: 1.24.0  
  
[Step 2]: Loading Harbor images ...  
Loaded image: goharbor/harbor-portal:v2.5.3  
Loaded image: goharbor/harbor-core:v2.5.3  
Loaded image: goharbor/redis-photon:v2.5.3  
Loaded image: goharbor/prepare:v2.5.3  
Loaded image: goharbor/harbor-db:v2.5.3  
Loaded image: goharbor/chartmuseum-photon:v2.5.3  
Loaded image: goharbor/harbor-jobservice:v2.5.3  
Loaded image: goharbor/harbor-registryctl:v2.5.3  
Loaded image: goharbor/nginx-photon:v2.5.3  
Loaded image: goharbor/notary-signer-photon:v2.5.3  
Loaded image: goharbor/harbor-log:v2.5.3  
Loaded image: goharbor/harbor-exporter:v2.5.3  
Loaded image: goharbor/registry-photon:v2.5.3  
Loaded image: goharbor/notary-server-photon:v2.5.3  
Loaded image: goharbor/trivy-adapter-photon:v2.5.3  
  
[Step 3]: preparing environment ...  
  
[Step 4]: preparing harbor configs ...  
prepare base dir is set to /root/resources/package/harbor
```

3. jke